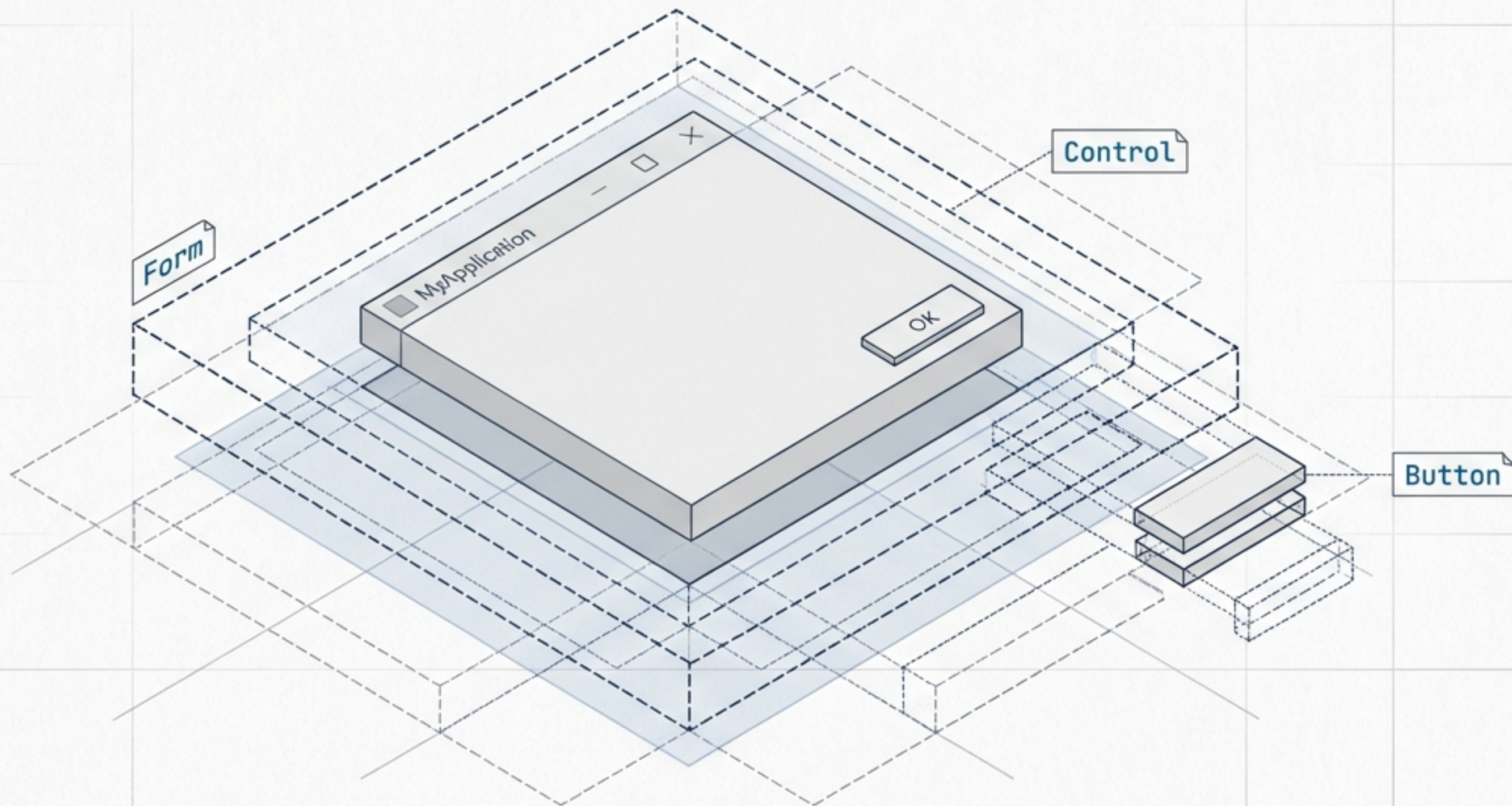


눈에 보이는 객체 지향

C# 윈도우 폼으로 완성하는 상속과 다형성의 청사진

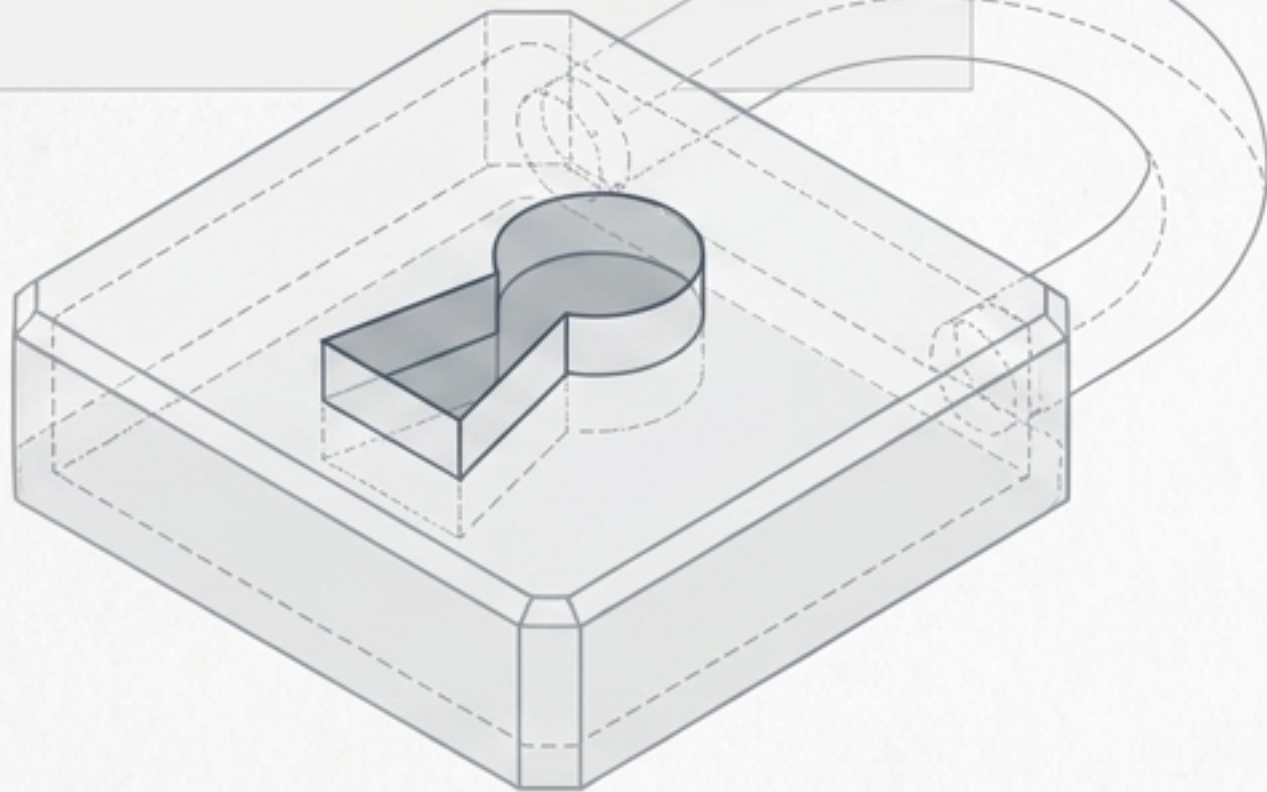


구현의 법칙: Abstract와 Override

추상(Abstract) 메서드는 프레임워크가 요구하는 '계약'입니다. 이 계약을 완수하려면 **override** 키워드를 통해 자식 클래스에서 구체적인 기능을 제공해야 합니다.

The Lock

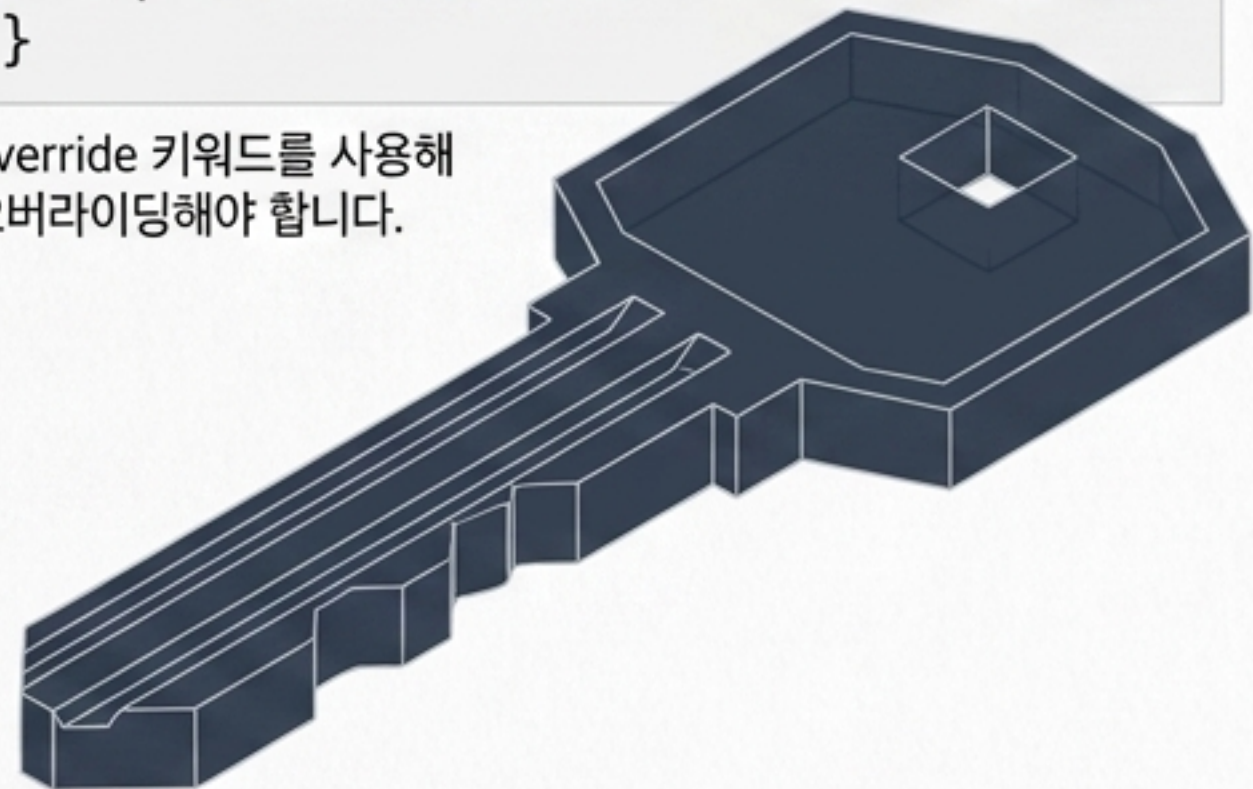
```
public abstract class Parent
{
    public abstract void Test();
}
```



The Key

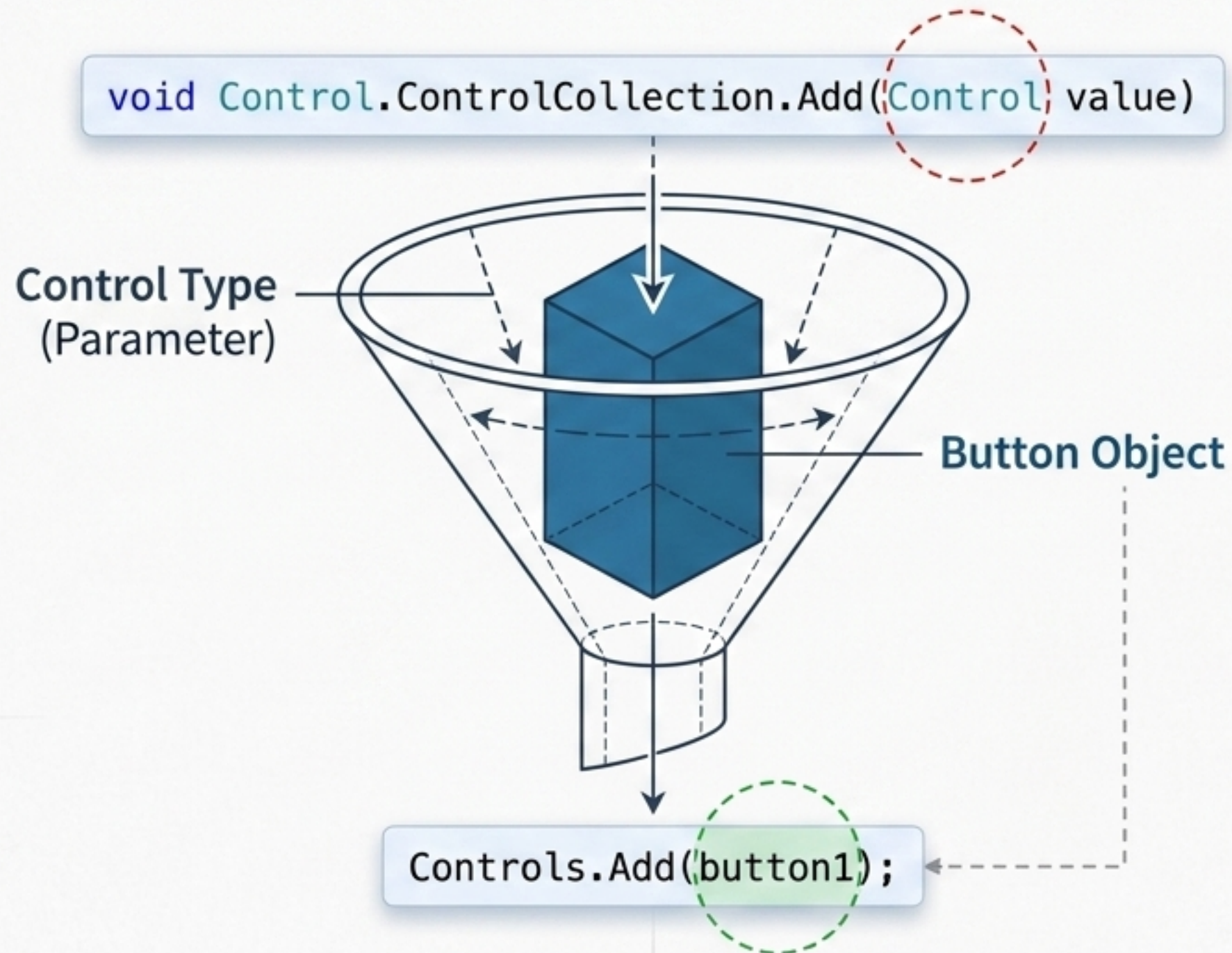
```
public class Child : Parent
{
    -- public override void Test() { }
```

override 키워드를 사용해
오버라이딩해야 합니다.



다형성의 퍼즐: 왜 Button이 Control 자리에 들어갈까?

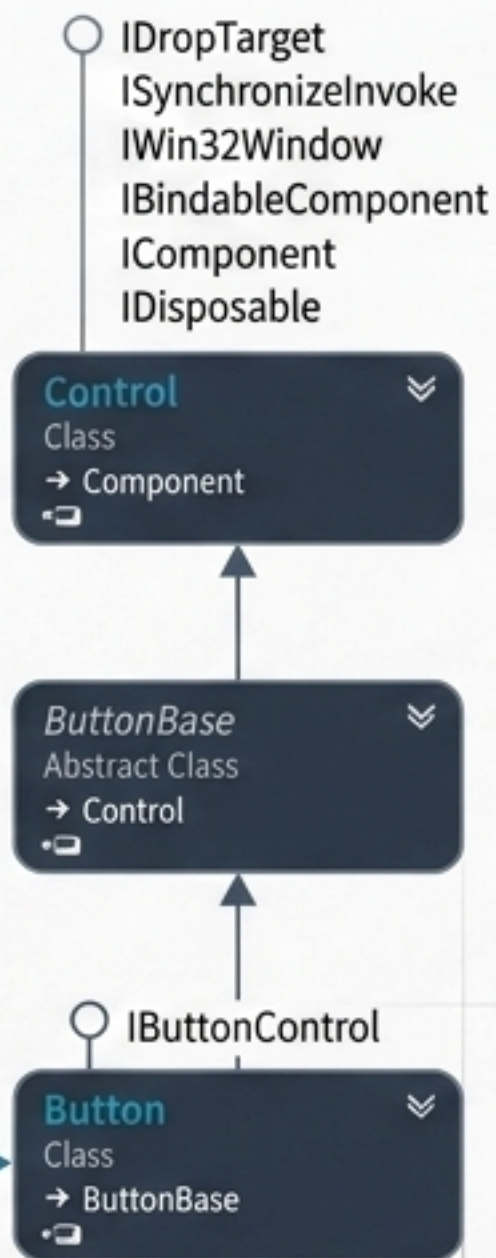
윈도우 폼 화면에 버튼을 추가할 때, 우리는 `Controls.Add()` 메서드를 사용합니다. 이 메서드는 분명 Control 인스턴스를 요구하지만, 우리는 Button을 넣습니다. 어떻게 이것이 가능할까요?



프레임워크 X-Ray: F12로 족보 추적하기

잘 모르는 프레임워크의 동작 원리는 직접 뜯어보아야 합니다. Visual Studio의 '정의로 이동(F12)' 기능은 클래스의 상속 관계를 투시하는 엑스레이(X-Ray)와 같습니다.

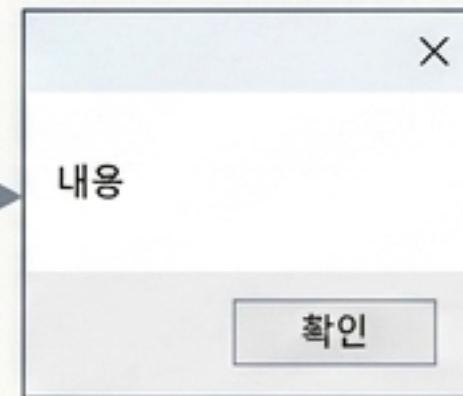
	디자이너 보기(D)	Shift+F7
	빠른 작업 및 리팩티링...	Ctrl+.
	이름 바꾸기(R)...	Ctrl+R, Ctrl+R
	Using 제거 및 정렬(E)	Ctrl+R, Ctrl+G
	코드 보기(C)	F7
	정의 피킹	Alt+F12
	정의로 이동(G)	F12
	기본으로 이동	Alt+홈
	구현으로 이동	Ctrl+F12
	모든 참조 찾기(A)	Shift+F12
	호출 계층 구조 보기(H)	Ctrl+K, Ctrl+T
	추적 값 원본	
	단위 테스트 만들기	
	중단점(B)	
	커서까지 실행(N)	Ctrl+F10
	커서까지 실행(R)	
	대화형으로 실행	Ctrl+E, Ctrl+E
	코드 조각(S)	
	잘라내기(T)	Ctrl+X
	복사(Y)	Ctrl+C
	붙여넣기(P)	Ctrl+V
	주석(A)	
	개요(L)	



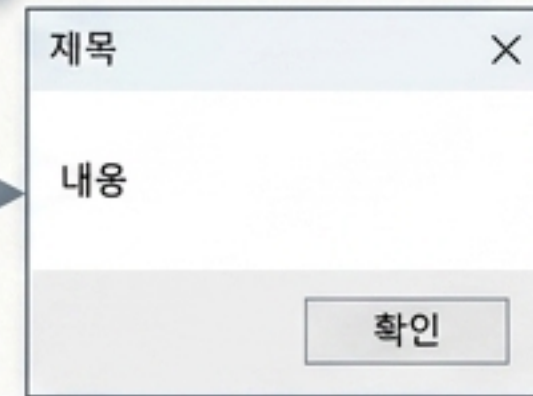
UI의 확장: 메서드 오버로딩 (Method Overloading)

MessageBox.Show()는 20가지 이상으로 오버로딩되어 있습니다.
매개변수를 추가할 때마다 메시지 상자의 UI 구조가 확장됩니다.

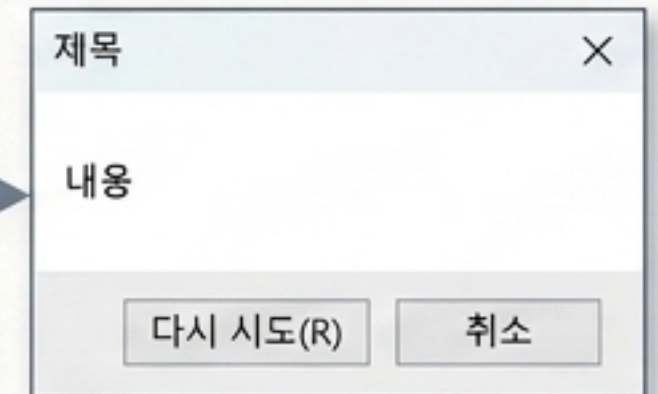
MessageBox.Show("내용");



MessageBox.Show("내용", "제목");

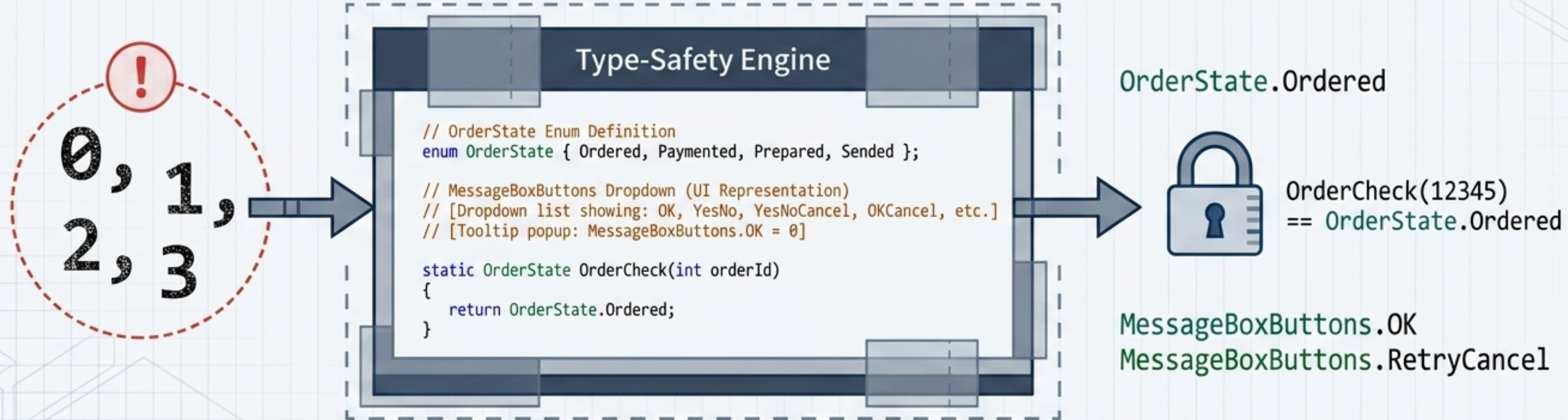


MessageBox.Show("내용", "제목", MessageBoxButtons.RetryCancel);



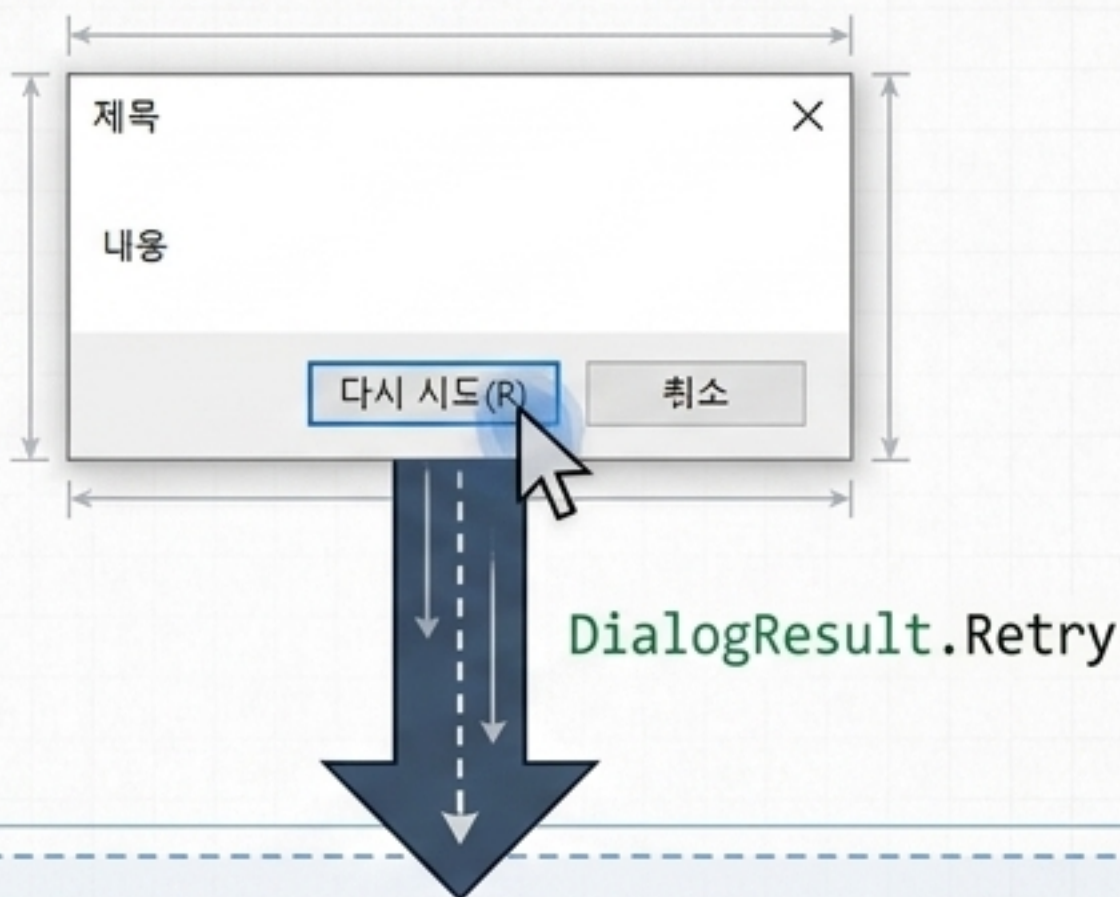
Enum (열거자): 안전한 코드를 위한 번역기

숫자에 명칭을 부여하는 enum 자료형은 프레임워크와 개발자 사이의 안전한 번역기 역할을 합니다. 단순한 정수 대신 의미 있는 이름을 사용하여 치명적인 오류를 방지합니다.



사용자와 코드의 대화: DialogResult와 흐름 제어

메시지 상자의 버튼 클릭은 단순한 화면 종료가 아닙니다. DialogResult 열거자를 통해 어떤 버튼이 눌렸는지 반환하며, 이를 do-while 문과 결합해 프로그램의 실행 흐름을 제어할 수 있습니다.

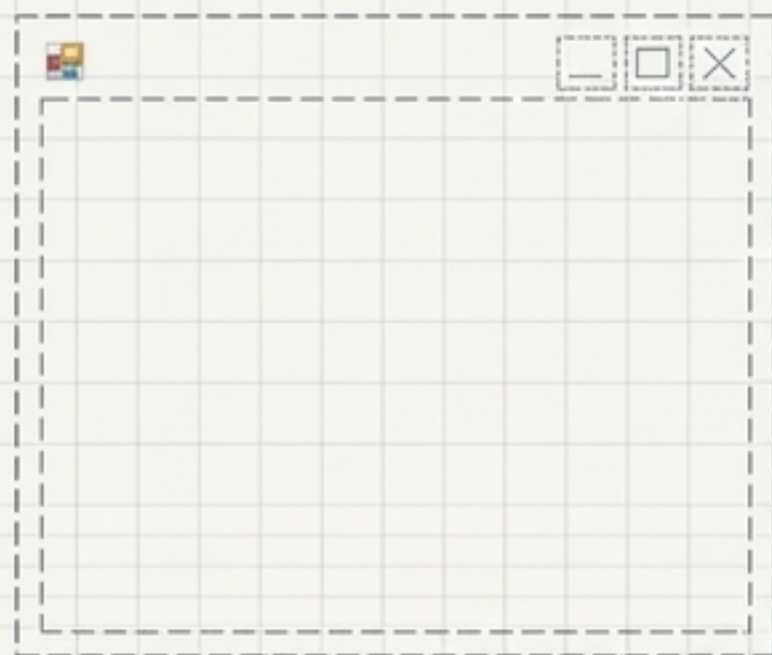


Click-to-Code

```
DialogResult result;  
07 do {  
09     result = MessageBox.Show("내용", "제목", MessageBoxButtons.RetryCancel);  
10 } while (result == DialogResult.Retry);
```

커스텀 아키텍처: Form 상속으로 화면 만들기

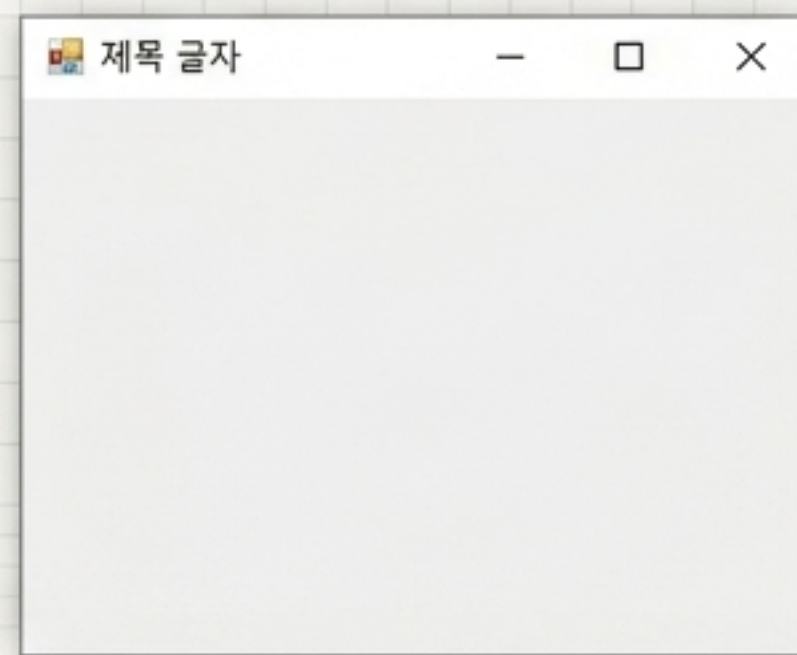
복잡한 화면 구성을 원한다면, 프레임워크가 제공하는 기본 Form 클래스를 상속(Inheritance)받아 자신만의 커스텀 폼 클래스를 설계해야 합니다.



Form
(Base Class)

```
class CustomForm : Form
```

상속
(Inheritance)



CustomForm

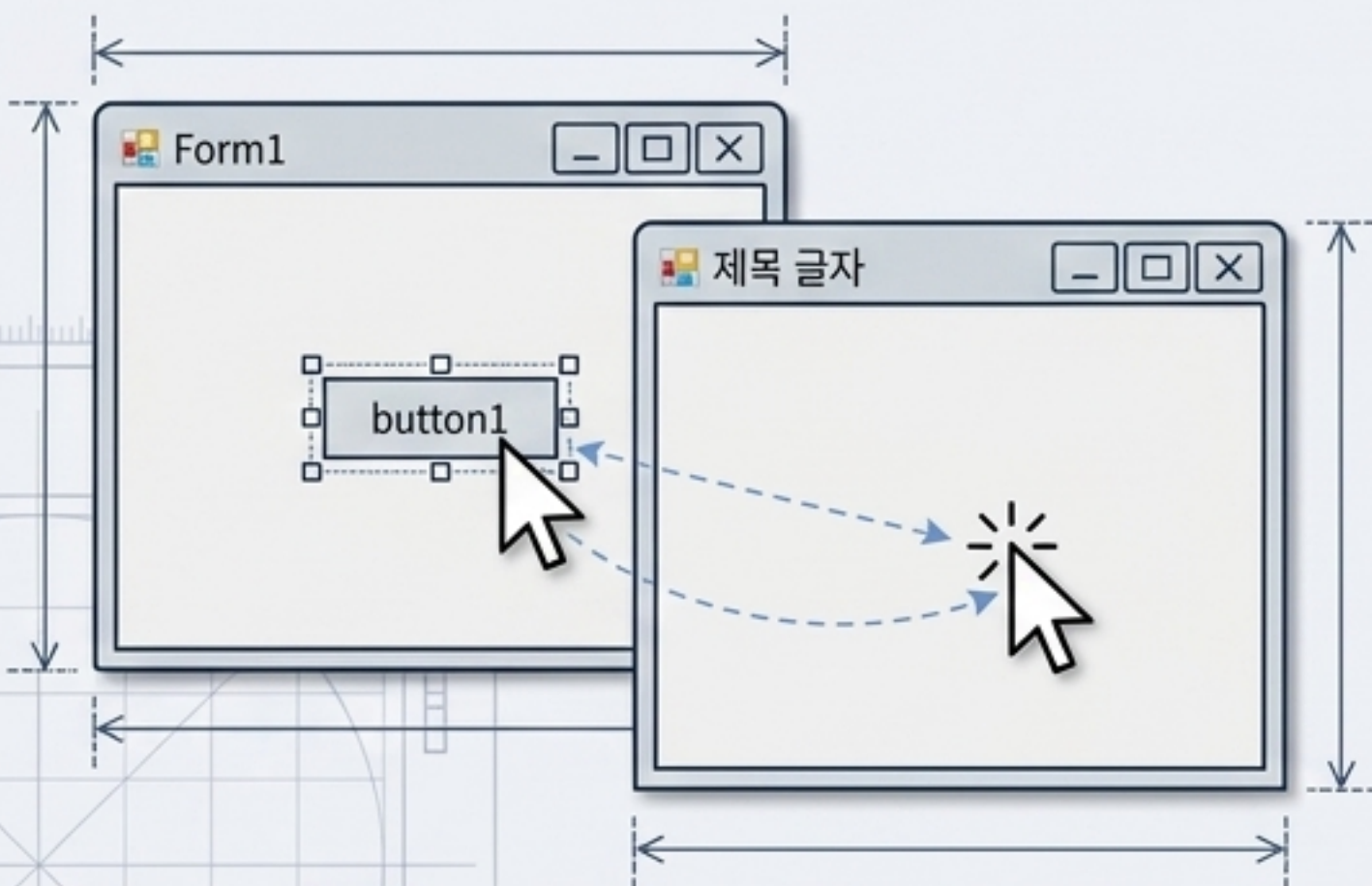
```
public CustomForm() {  
    Text = "제목 글자";  
}
```

창의 동작 제어: Modal vs. Modeless

상속으로 만든 폼을 화면에 띄울 때, 메서드 선택 하나가 애플리케이션의 동작 방식(상호작용 가능 여부)을 완전히 바꿉니다.

Modeless (모달리스) - The Post-it Note

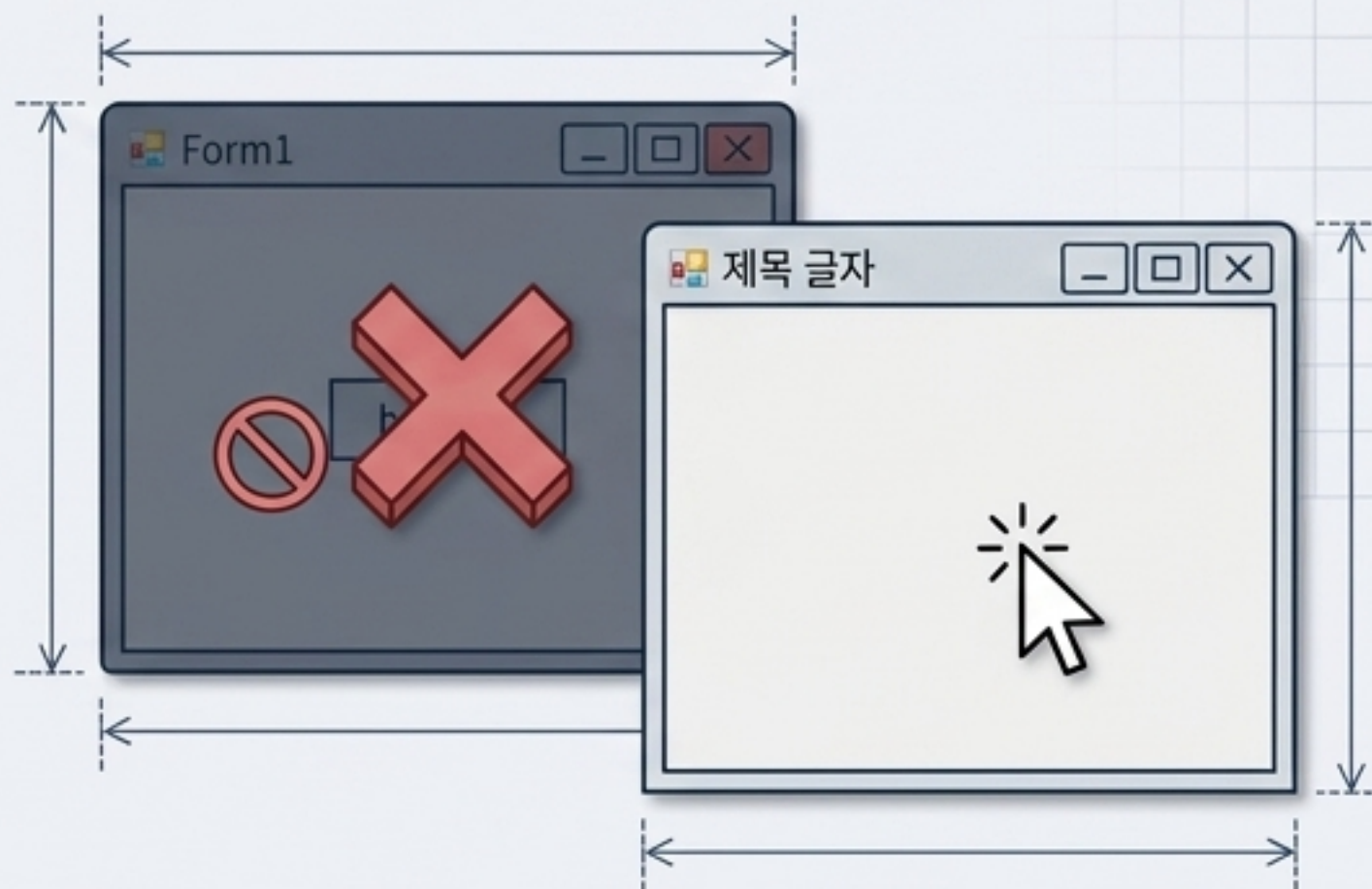
```
14 CustomForm form = new CustomForm();  
15 form.Show(); // Form 클래스를 상속받습니다.
```



두 창 모두 자유롭게 상호작용 가능 (Free Interaction)

Modal (모달) - The Bouncer

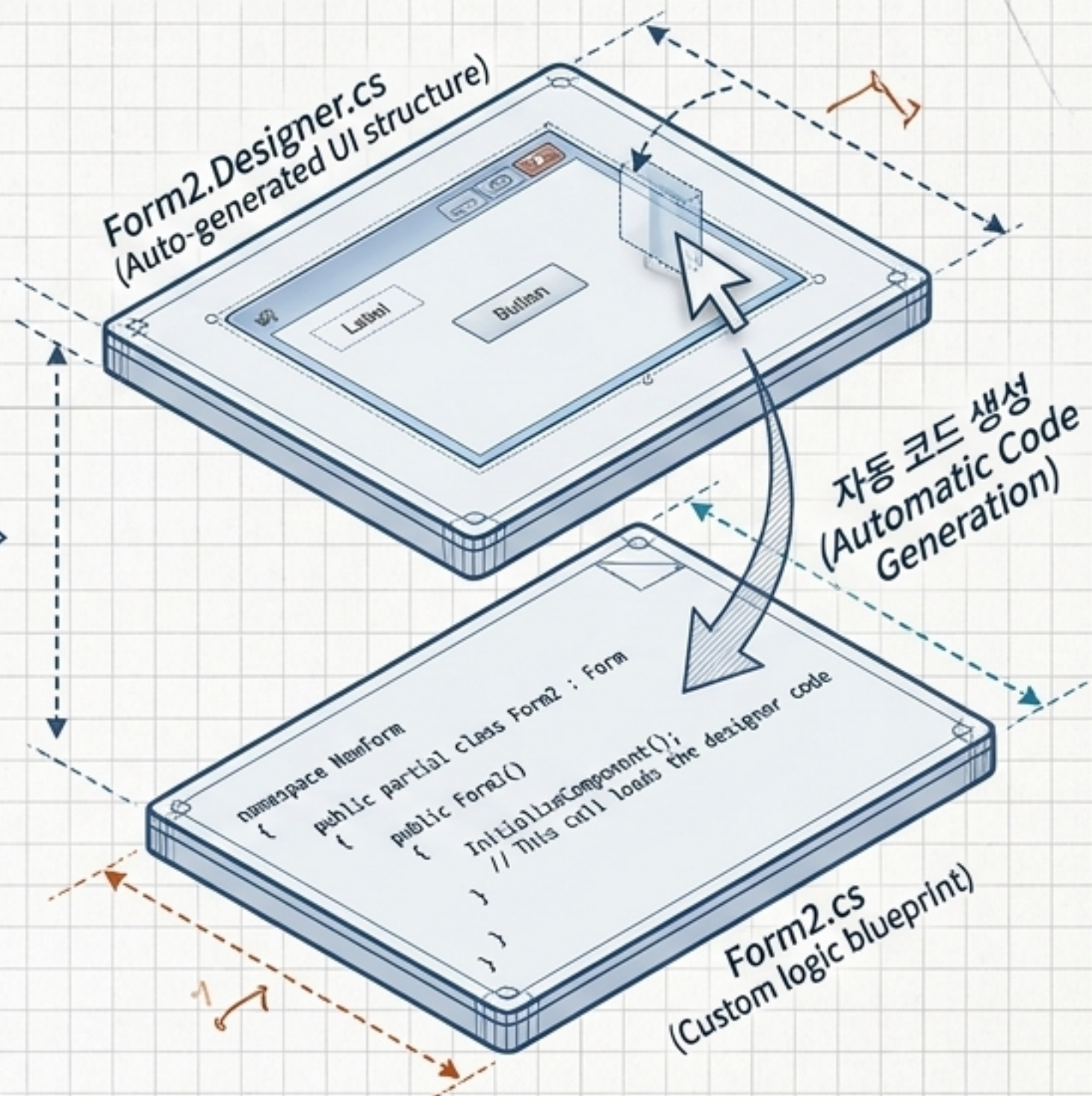
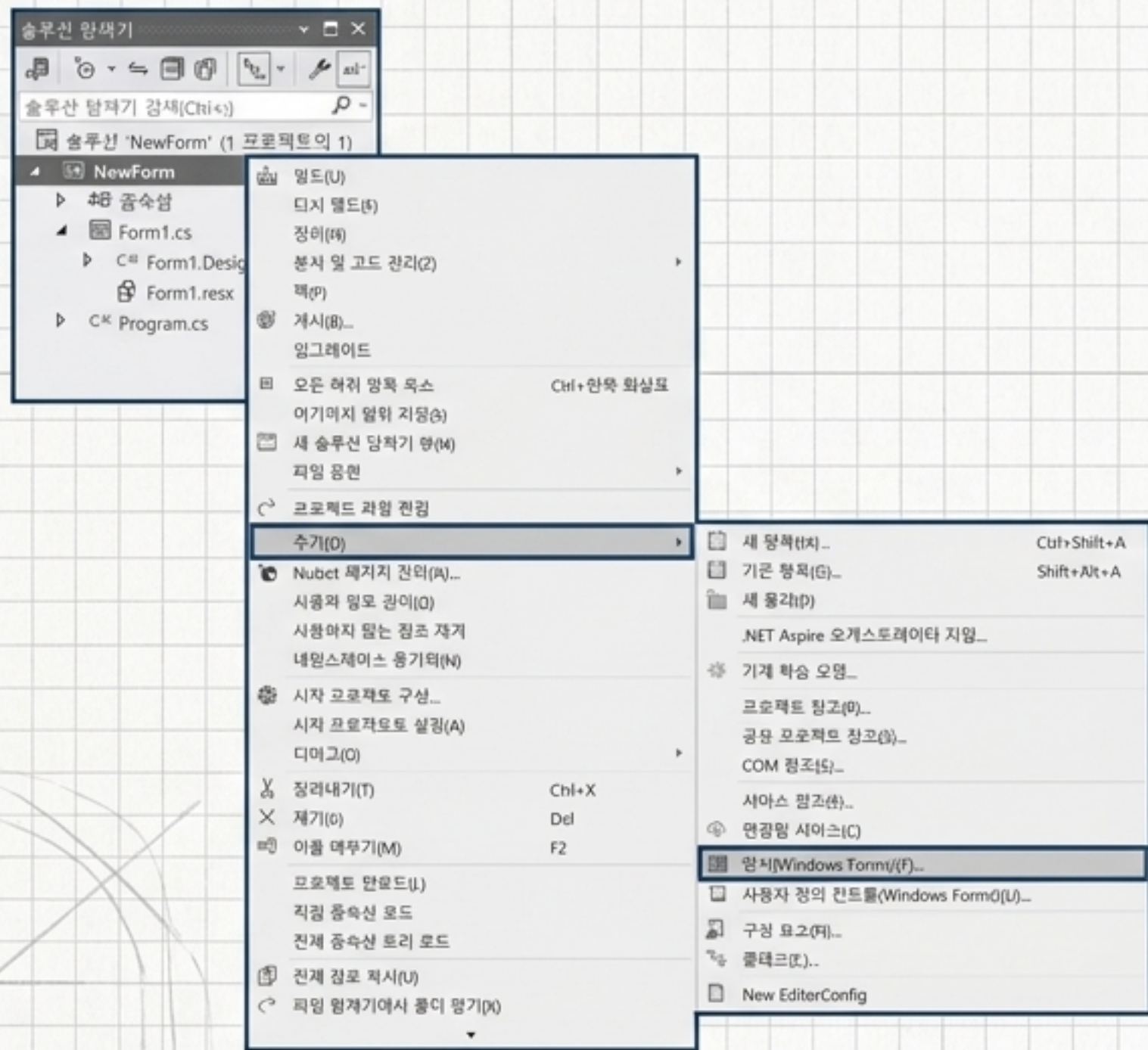
```
03 CustomForm form = new CustomForm();  
04 form.ShowDialog(); // 모달 대화상자를 생성합니다.
```



배경 창 상호작용 불가 (Background Interaction Blocked)

개발자 워크플로우: IDE가 작성하는 자동 청사진

실무에서는 코드로 모든 것을 타이핑하지 않습니다. Visual Studio의 디자이너 도구는 [추가] - [양식(Windows Form)] 메뉴를 통해 Form 상속과 파일 생성을 자동으로 처리합니다.



Grand Synthesis: UI 속에 숨겨진 OOP 메커니즘

다형성, 상속, 오버로딩, 열거자. 이 모든 객체 지향의 추상적인 개념들은 단순히 교과서에 머물지 않습니다. 우리가 매일 클릭하고 상호작용하는 윈도우 UI를 지탱하는 뼈대이자 신경망입니다.

